assignment-02\task-2b-bpe.py

```python
# Task 2 b) Byte Pair Encoding (BPE)

'''
Genutzte Ressourcen:
Umsetzung Code und Programmstruktuer:
https://towardsdatascience.com/byte-pair-encoding-for-beginners-708d4472c0c7/

Fachliche und wissenschaftliche Quelle
Rico Sennrich, Barry Haddow, and Alexandra Birch.
2016. Neural Machine Translation of Rare Words with Subword Units.
arXiv preprint arXiv:1508.07909v5. https://arxiv.org/abs/1508.07909

'''

import collections
import re
import json

# Anzahl Iterationen
num_merges = 15
corpus_file = "assignment-02\\corpora-de.json"
unknown_file = "assignment-02\\unknown-de.txt"

def read_corpora(input_file):
    vocab = {}
    with open(input_file, "r", encoding="utf-8") as file:
        vocab = json.load(file)
        return vocab

def read_unknown(input_file):
    with open(input_file, "r", encoding="utf-8") as file:
        unknown_text = file.read()
        return unknown_text


# Häufigkeit benachbarter (Trenner=" ") Elemente zählen
def get_stats(vocab):
    pairs = collections.defaultdict(int)
    for word, freq in vocab.items():
        elements = word.split()
        for i in range(len(elements)-1): # -1, weil i noch einen Nachfolger haben muss.
            pairs[elements[i], elements[i+1]] += freq
    return pairs

# Benachbarte Elemente verschmelzen, basierend auf den Elementen im Korpus
def merge_vocab(pair, vocab_in):
    vocab_out = {}
    # Suchtext
    search_text = ' '.join(pair)
    # Ersatztext
    replace_text = ''.join(pair)
```

```
52      # Sonderzeichen maskieren
53      escaped_search_text = re.escape(search_text)
54      # Suchmuster
55      pair_pattern = re.compile(r'(?<!\S)' + escaped_search_text + r'(?!\S)')
56
57      for word, frequency in vocab_in.items():
58          new_word = pair_pattern.sub(replace_text, word)
59          vocab_out[new_word] = frequency
60
61      print(f"vocab_out {vocab_out}")     # debug print
62      return vocab_out
63
64  # Beste Paarung, zuerst im Alphabet
65  def best_alphabetical(pairs):
66      # Paar mit höchtem Wert = Anzahl
67      max_count = max(pairs.values())
68      if max_count == 1:
69          print("ACHTUNG! Nur noch je 1 Paar vorhanden!")
70      # Alle Paare mit höchstem Wert finden
71      all_bests = []
72      all_bests_concat = []
73      for pair in pairs:
74          if pairs[pair] == max_count:
75              all_bests.append(pair)
76              all_bests_concat.append("".join(pair))
77      # Erstes Paar im Alphabet finden
78      best_index = all_bests_concat.index(min(all_bests_concat))
79      best = all_bests[best_index]
80      print(f"max_count: {max_count} all_bests: {all_bests} best: {best}") # debug print
81      return best, all_bests
82
83  # Token-Liste
84  def current_token_list(vocab):
85      vocab_list = list(vocab)
86      current_list = []
87      for word in vocab_list:
88          tokens = word.split()
89          for token in tokens:
90              if token not in current_list:
91                  current_list.append(token)
92      return current_list
93
94  def learn_bpe(vocab, num_merges):
95      for i in range(num_merges):
96          print()         # debug print
97          print("i:", i)  # debug print
98          # Häufigkeit benachbarter, durch " " getrennte Elemente, zählen
99          pairs = get_stats(vocab)
100         # Häufigste Paarung (alphabetisch) in best ablegen
101         best, all_bests = best_alphabetical(pairs)
102         # Häufigsten Elemente verschmelzen, gibt das Ergebnis als neuen Korpus zurück
103         vocab = merge_vocab(best, vocab)
104         token_list = current_token_list(vocab)
105         print(f"token_list {token_list}")
```

```
    return vocab, token_list



# Mit KI-Unterstützung generierter Code -- Anfang

# Ein Wort mit Hilfe der Token-Liste in Tokens zerlegen
def segment_word(word, token_list):
    tokens = []      # hier sammeln wir die gefundenen Tokens
    position = 0     # aktuelle Stelle im Wort

    # So lange wiederholen, bis das ganze Wort zerlegt ist
    while position < len(word):
        found_token = None

        # Vom längstmöglichen Stück zum kürzesten probieren
        for end in range(len(word), position, -1):
            piece = word[position:end]
            if piece in token_list:
                found_token = piece
                break    # längstes passendes Token gefunden, Suche beenden

        # Kein Token passt: einzelnes Zeichen als Token nehmen
        if found_token is None:
            found_token = word[position]

        tokens.append(found_token)
        position = position + len(found_token)   # hinter dem Token weitermachen

    return tokens


# Einen ganzen Korpus (Text) segmentieren
def segment_corpus(text, token_list):
    result = []
    words = text.split()            # Text an Leerzeichen in Wörter teilen
    for word in words:
        word_tokens = segment_word(word, token_list)
        result.append(" ".join(word_tokens))   # Tokens mit Leerzeichen verbinden
    return result
# Mit KI-Unterstützung generierter Code -- Ende



# Hauptroutine
# Token-Liste, basierend auf Korpus erstellen
print(f"num_merges: {num_merges}")            # debug print
vocab = read_corpora(corpus_file)
print(f"corpus: {vocab}")
vocab, final_token_list = learn_bpe(vocab, num_merges)
print()                                       # debug print
print(f"final_token_list {final_token_list}") # debug print

# BPE Encoding auf unbekannten Korpus anwenden
```

```
# Beispielaufruf
unknown_text = read_unknown(unknown_file)
segmented = segment_corpus(unknown_text, final_token_list)
print(f"segmented: {segmented}")


```